

Contents/interpreter

1. Lexical analysis
2. Syntax analysis
3. AST
4. Semantic analysis
5. Linker/namespaces

1.Lexer

The source code is checked for whether the symbols included in it are acceptable by Snake and if they are acceptable, they are converted to a list of tokens, so that the interpretation process can continue. I chose to use tokens, as they make parsing much easier, as they can be easily converted to context free grammar symbols, which are used by the LL(1) parser. The tokens themselves are useful too, as the CFG symbols omit a lot of information (ex variable names), so I use both in the parsing and AST creation phase. Also, during the lexing stage, Strings are checked for whether they are declared correctly. For example

```
string<- 'hello' is correct and becomes [TVar "string",TAssign,TQuo,TStr  
'hello',TQuo]
```

```
While
```

```
string<- 'hello is incorrect and an error occurs.
```

2.Parser

First, the first/follow sets are created, then using those the parsing table is creating and using that, every given line of tokens gets converted into grammar symbols and goes through the recursive predictive LL(1) parser (parser0 function in my code), that consults the parsing table to decide which production to apply. When rules are applied, they leave markers so that the Abstract Syntax Tree can be created and terminals that are popped are saved in the AST. If the top of the stack is a terminal and the next input symbol is the same type of terminal it is popped, if the stack has a non terminal on top, then grammar rules are applied based on the parsing table. If the terminal on top of the stack cannot be matched with the next input terminal, a syntax error occurs. Same for when the combination of the top of the stack non terminal and the next input terminal cannot be found in the parsing table.

3.AST

ASTs are created recursively and they represent a line/part of code that then gets interpreted. They are represented using a data type that allows the connection of multiple ASTs into one, so they can be built recursively:

```
data AST
```

```

= Assign AST AST
| Read AST
| Print AST
| Add AST AST
| Sub AST AST
| Mul AST AST
| Div AST AST
| Var String
| Num Int
| Str String
| F String
| FunctDef AST AST AST
| Funct AST
| Array [AST]
| If AST AST AST
| Then AST
| Else AST
| Do AST AST
| Block [AST]
| Less AST AST
| Greater AST AST
| Equal AST AST
| Free AST
| Boolean Bool
| Param [AST]
| Return AST
| Length AST
| Import AST
| Random AST
deriving (Show, Eq)

```

AST example:

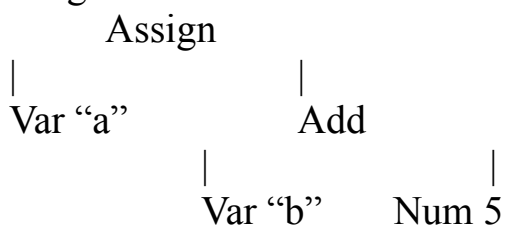
The statement

`a <- b + 5`

Becomes

`Assign (Var "a") (Add (Var "b") (Num 5))`

Diagram:



I chose to create the ASTs during parsing, so the parsing can be done in one pass, combining checking the syntax of a given statement(s) and creating the syntax tree, that is evaluated during the semantic analysis.

4. Semantic analysis

Values of expressions are evaluated recursively using the different eval functions in my code, where the different parts of the expressions need to be of an acceptable type, so that they can be meaningfully evaluated. Statements are interpreted using my compile functions, based on the statement of each AST (assign, if statements, print, read etc) and the rules of the language that dictate how they should be handled.. The non in-built functions (read , print , length , random and return) are saved in the function table or in the module table, if they are imported from a library. The variables are saved in the value table, which includes their values, that can change as the program is interpreted. I used ordered maps to implement them, as some times (for example for discovering is a function returns a result and finding it, as I decided to not declare function types, so that they can be easily overloaded), multiple maps need to be searches, which is done more efficiently using ordered maps in contrast to unordered maps ($O(\log n)$ vs $O(n)$ for unsuccessful searches). The functions, are saved as their respective ASTs, which include their statement(s) and their parameters and their ASTs are interpreted the moment they are called. The parameter matching is done by the match function, which takes a list of the parameters of the function when it is called and the parameters of the function when it is declared and they are matched (obviously they need to be equal in number, else an error is going to be created). A function can only use variables that have been passed to it as parameters or have been declared inside the function during its current call.

5. Linker

Whenever the import function is used, the whole module is imported and saved in the module table. Due to Haskell's lazy compilation, the functions inside the module are not interpreted right away, but rather on the first call of a function from that module. Whenever an imported function is called, the name of the module is located and the module gets found in the module table. After that, the function is found in the module and (after the module is interpreted), gets executed. The modules themselves have an ordered map of functions each, so that the correct function can be located quickly.

6. Walkthrough the whole process

Statement: `a<- random 5`

Lexer: `[TVar "a",TAssign,TRand,TNum 5]`

AST created from parser: `Assign (Var "a") (Random (Num 5))`

Interpretation: `Random (Num 5)` gets evaluated first, where after `Num 5` is evaluated as a value (`Int 5`), it gets sent to the `Random` function, which produced a random number $0 \leq \text{number} < 5$. Then, this number gets assigned

to the variable “a” and the value table (map in my code) gets updated, as the pair: Key = “a”, Value = (random number from 0 to 4) is inserted into it.

Statement: print a * 2

Lexer: [TPrint,TVar "a",TMul,TNum 2]

AST: Print (Mul (Var "a") (Num 2))

Interpretation: error: Variable "a" not defined or not in scope

The variable was a not defined before being used, so an error was produced.